Ders 2 Genişletilmiş Notu: Tasarım Kalıplarına Giriş, MVC ve Abstract Factory Kalıbı

Ders 2 Genişletilmiş Notu: Tasarım Kalıplarına Giriş, MVC ve Abstract Factory Kalıbı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tasarım Kalıbı Nedir?
- 2. Bir Tasarım Kalıbının Bileşenleri
- 3. Tasarım Kalıplarının Yararları
- 4. Kalıpların Sınıflandırılması ve Anlatım Sırası
- 5. Model-View-Controller (MVC) Kalıbı
- 6. Abstract Factory Kalıbı
- 7. Abstract Factory Örneği: Üç Boyutlu Oyun Motoru
- 8. Örnekten Çıkarılan Tasarım İlkeleri
- Örnek Sorular
- Dersin Toparlaması

1. Tasarım Kalıbı Nedir?

Dersin büyük bölümü tasarım kalıpları üzerine kuruludur. Hocanın slaytındaki tanıma göre tasarım kalıbı, yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir (ekrandan, 00:11:10). Çözüm somut bir gerçekleme olmak zorunda değildir; soyut bir tarif de olabilir.

Tasarım kalıplarının nesneye yönelik olması zorunlu değildir. Bu konudaki ilk temel katkıyı yapan Dörtlü Çete (Gang of Four) kalıplarını nesneye yönelim üzerinden önerdiği için akla ilk olarak nesneye yönelim gelir. Ama farklı programlama yaklaşımları ya da mimari stiller için de onlara özgü kalıplar önerilebilir.

Bir kalıbın önerisindeki ayrıntı düzeyi çok değişebilir. Örneğin bu derste ilk görülecek kalıp (MVC) hiç kaynak kod içermez; bir öneri gibi de düşünülebilir. İlerideki kalıplar ise çok daha ayrıntılı yönlendirme yapar. Hocanın vurgulamak istediği, bir kalıbın çok ayrıntılı olmasının gerekmediği, önemli olanın aşağıda anlatılan bileşenleri içermesidir.

Amaç, tekerleği yeniden keşfetmeden “iyi” tasarıma ulaşmaktır: sorunları esnek biçimde ve yeniden kullanılabilirliği artıracak yönde çözmek. Esneklik önemlidir, çünkü hocanın deyişiyle bu meslekte değişmeyen tek şey değişimin kendisidir. Gereksinimler mutlaka değişir ve bu değişikliklerin bazı nedenleri bizim elimizde değildir. Tasarım kalıpları, proje sırasında ve destek süresince yapılması gereken zorunlu değişiklikleri en az çabayla ve tasarımın genel kalitesini bozmadan yapabilmeyi sağlar.

1.1 Terimin kökeni

Bilgisayar mühendisliği nispeten yeni bir meslek olduğu için başka mesleklerden terimler ödünç alıp kendi bağlamına uyarlar. “Tasarım kalıbı” da mimarlıktan gelir. Mimaridеki bir kalıp örneği: içinde bir insanın günde bir saatten fazla vakit geçireceği her yerde mutlaka bir

pencere bulunmalıdır. Nesneye dayalı tasarım ve modellemede de kalıplar, benzer biçimde iyi bir çözüm öneren tavsiyelerdir.

2. Bir Tasarım Kalıbının Bileşenleri

Bir tasarım kalıbının dört ana bileşeni vardır (ekrandan, 00:11:10):

- İsim: kalıbı en güzel şekilde anlatacak, bir ya da birkaç kelimelik ad.
- Sorun: kalıbın çözdüğü sorunun tarifi.
- Çözüm: sorunu çözmek için önerilen tasarımın bileşenleri; bu bileşenlerin ilişkileri, sorumlulukları, birlikte çalışmaları.
- Tartışma/Sonuçlar: çözümün sistemin geneline esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden yaptığı etkiler; çözümün güçlü ve zayıf yönleri; yer-zaman çelişkileri; başka kalıplar ile işbirliği olanakları.

2.1 İsim

Bir şeye iyi bir isim vermek kolay değildir. İyi isim, kalıbın özünü çok iyi yansıtmalıdır: kalıbın içeriği okunduktan sonra "ne kadar uygun bir ad" dedirtmeli ve kalıba bir kez aşina olunduktan sonra yalnız adından kalıpla ilgili her şeyi hatırlatabilmelidir. Hoca, MVC'nin adının şu an pek bir şey ifade etmeyebileceğini, ama kalıp işlendikten sonra adın ne kadar uygun olduğunun görüleceğini söyledi.

2.2 Sorun ve çözüm

İsimden sonra kalıbın çözdüğü sorun tarif edilir; çoğu zaman bu sorunun neden zor olduğuna da değinilir. Ardından en belirgin kısım olan çözüm gelir. Çözüm sınıflar, nesneler, üye alanlar, metotlar ve aralarındaki ilişkiler biçiminde tarif edilir ve UML sınıf şemalarıyla belgelenir.

2.3 Tartışma ve sonuçlar

Son bölümde, kalıp iyi bir çözüm sunsa da mükemmel çözüm diye bir şey olmadığı kabul edilir. Her çözümün avantajları ve dezavantajları vardır; önerilen tasarım gerçeklenirken de farklı kararlar alınabilir ve bunların her birinin getirdikleri ve götürdükleri tartışılır. Kalıbın esneklik, genişletilebilirlik, taşınabilirlik gibi iç kalite özelliklerine (yani kodlayanı ilgilendiren özelliklere) etkisi anlatılır.

Yer-zaman çelişkisi (time-memory trade-off) meslekte önemli bir denge noktasıdır: bir hesaplama daha fazla bellek harcanarak daha kısa sürede yapılabilir, ama bunun bir dengesi gerekir. Ayrıca kalıbın başka kalıplarla olası işbirlikleri anlatılır. Hocanın örnek cümlesi: "Her bir Strategy gerçeklemesi bir Singleton olabilir." Bu bilgi, büyük bir problemin bir kısmı için uygun bir kalıp bulunduğunda sonraki adımların neler olabileceğini hızlıca gösterir.

2.4 Kalıplar arası ilişki şeması

Hoca, Dörtlü Çete'nin kitabından alınmış, kalıplar arasındaki ilişkileri ve birlikte çalışma olasılıklarını gösteren şemayı gösterdi (ekrandan, 00:16:50). Şemadaki ilişkilerden bazıları:

- Abstract Factory: Factory Method ile gerçeklenebilir (implement using), Prototype ile dinamik olarak yapılandırılabilir (configure factory dynamically), tek örnek (single instance) için Singleton kullanılabilir.
- Facade: tek örnek için Singleton kullanılabilir.
- Template Method: Factory Method'u sık kullanır (often uses), Strategy'de algoritma adımlarını tanımlar.
- Composite: Builder ile oluşturulabilir, Iterator ile çocukları dolaşılabilir, Decorator ile nesnelere sorumluluk eklenebilir, Visitor ile işlem eklenebilir, Flyweight ile paylaşılabilir.
- Observer ile Mediator arasında "karmaşık bağımlılık yönetimi" ilişkisi vardır.

Hocaya göre olası ilişkiler yalnız bunlar değildir. Kalıplar farklı problemlere uyarlandıkça, örneğin şemada doğrudan ilişkisi olmayan Chain of Responsibility ve Command'ın bir problemde

birlikte çalışması gerekebilir. Şemanın ezberlenmesi kesinlikle gerekmez; ama kalıplarla çalışırken bir kenarda durması, problemin geri kalanını nasıl modellemek gerektiği konusunda yararlı olur.

3. Tasarım Kalıplarının Yararları

Hocanın saydığı yararlar şunlardır (ekrandan, 00:20:00):

- Tekerleği yeniden icat etmemek: olgunlaşmış çözüm önerileri vardır; bunları sıfırdan keşfetmek yerine varlıklarını bilmek zaman kazandırır.
- Kaliteli çözüm: kalıpların üzerinde birçok deneyimli kişinin emeği geçmiştir.
- Deneyimin yazılı biçimde aktarılabilmesi: deneyim çoğunlukla yaparak kazanılır ve aktarılır; kalıplar deneyimli kişilerin birikimini yazılı olarak aktarır. Bu, özellikle nesneye dayalı tasarıma yeni başlayanlara hızlı bir başlangıç (kick-start) sağlar. Hoca bunu Matrix filminde bir tuşa basılarak insanların beynine dövüş sanatları ya da helikopter kullanma bilgisinin yüklenmesine benzetti.
- Yazılım ekibi arasında ortak bir sözlük oluşturması.

Son yarar ilk bakışta akla gelmeyebilir. Gereksinim belgesinden yola çıkılarak modelleme yapılırken kıdemli bir tasarımcı "problemin şu kısmında Strategy kalıbını kullanalım" der; öteki "haklısın, buradaki stratejilerde Template Method da kullanmamız iyi olur, çünkü yanlış yapılabilecek birçok kısım var" diye ekler. İki cümleyle sistemin önemli bir kısmının nasıl tasarlanacağı ekibe anlatılmış olur. Uzun uzun çizmek ve anlatmak yerine iki-üç kalıp adı vererek tartışmak, anlaşmak ve uzlaşmak mümkündür. Bu nedenle hocaya göre tasarım kalıpları yazılım alanında çalışan herkesin bilmesi gereken bir konudur.

Birinin kalıp diye nitelendirdiği bir tavsiyeyi bir başkası "bu temel bir ilke" deyip geçebilir. Hocaya göre her iki taraf da kendi görüşüne göre haklıdır, çünkü bir kalıbın çok ayrıntılı olması gerekmez. Ama iyi bir fikir her zaman iyi bir fikirdir ve küçümsenmemelidir.

4. Kalıpların Sınıflandırılması ve Anlatım Sırası

4.1 Amaca göre üç küme

Ders notlarındaki GoF kalıpları, kaynak kitaptaki sıralamaya sadık kalınarak anlatılır. Kitap kalıpları amaçlarına göre üç kümeye ayırır (ekrandan, 00:22:45):

- Creational (yaratımsal): nesnelerin oluşturulması, temsili ve ilişkilendirilmesindeki bağımlılığı (coupling) azaltmaya yönelik kalıplar. Hocanın eklediği ayrıntı: bu kalıplar bağımlılığı olması gerekenin altına indirmez, olması gerekenden yüksek olmasını engeller.
- Structural (yapısal): sınıflar ve nesnelerin bir araya getirilerek daha büyük ve karmaşık yapılar elde edilmesine yönelik kalıplar.
- Behavioral (davranışsal): sorumlulukların nesnelere dağıtılmasına ve algoritma seçimine yönelik kalıplar. Hoca, algoritma seçiminin çalışma zamanında (runtime, "on the fly") yapılabilmesine izin verecek tasarımları vurguladı.

Hangi kalıbın hangi kümeye girdiğini ezberlemek gerekmez; önemli olan kümelerin çözmeye çalıştığı problemlerin ortak noktalarını bilmektir.

4.2 Odağa göre iki grup: sınıfsal ve nesnesel kalıplar

Dörtlü Çete kalıpları ayrıca odaklarına göre ikiye ayırır (ekrandan, 00:30:00):

- Sınıfsal kalıplar: sınıflar ve bunların alt sınıfları arasındaki ilişkilere yöneliktir. Sınıf yapısı ve kalıtım ilişkisinin doğası gereği bu ilişkiler derleme zamanında belirlenir ve daha durağandır.
- Nesnesel kalıplar: çeşitli türden nesneler (aynı ya da farklı sınıflardan) arasındaki ilişkilere yöneliktir. Bu ilişkiler kodun çalışması süresince rahatlıkla değiştirilebilir ve daha dinamiktir.
- Adapter kalıbı GoF tarafından hem sınıfsal hem nesnesel olarak değerlendirilmiştir. Diğer kalıplar bu iki gruptan yalnız birine dahildir.

Kalıtımın durağanlığını hoca şöyle örnekledi: öğrenci sınıfı insan sınıfının alt sınıfıysa her öğrenci her zaman bir insandır; başka bir alternatif düşünülemez. Buna karşılık "bir insanın bir arabası vardır" gibi bir sahiplik ilişkisinde belirli bir insanla belirli bir araba arasındaki ilişki değiştirilebilir; insanın arabasını başkasına satması modellenebilir, sahiplik bir nesneden alınıp diğerine verilir. Oysa insan sınıfından öğrenci ve çalışan sınıfları türetilmişse, bir öğrenci nesnesi doğrudan bir çalışan nesnesine dönüştürülemez; durum bilgisini çalışan sınıfına aktaracak kopya kurucu (copy constructor) gibi ek kodlar gerekir.

> **Hocanın vurgusu:** Kalıtım güçlü bir araçtır ve tam kullanılması gereken yerlerde çok işe yarar. Ama yanlış kullanımı çok büyük zararlar doğurabilir. Kalıp örneklerinde akla ilk gelen çözüm genelde kalıtımı doğru kullanmayan bir çözüm olacak ve en ufak bir değişiklik karşısında ne kadar çok iş çıkardığı görülecektir.

4.3 Derste izlenen sıra

Hoca, kalıpları listeleyen bir tabloyu gösterdi (ekrandan, 00:31:05). Tablodaki sıra kitabın anlatım sırasıyla hemen hemen aynıdır: MVC (davranışsal olarak etiketlenmiş), ardından yaratımsal kalıplar (Abstract Factory, Builder, Factory Method, Prototype, Singleton), yapısal kalıplar (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) ve davranışsal kalıplar (Chain of Responsibility, Command, Iterator, Memento, Observer, State, Strategy, Template Method ve diğerleri). Interpreter kitapta 15. sırada anlatılır; derste sona alınmıştır.

> **Ders sonrası not:** Tabloda MVC davranışsal bir kalıp olarak etiketlenmiştir. Dörtlü Çete kitabı MVC'yi 23 kalıplık kataloğunda ayrı bir kalıp olarak vermez; giriş bölümünde Smalltalk MVC'yi Observer, Composite ve Strategy kalıplarıyla açıklar. Kitabın kalıp anlatımına MVC ile başladığı söylemi bu giriş bölümünü kastediyor olmalıdır (büyük olasılıkla).

4.4 Yedi gruplu sınıflama

Dörtlü Çete kitabında bir de yedi gruplu bir sınıflama vardır (ekrandan, 00:32:35). Hoca eskiden kalıpları bu sıraya göre anlatıyordu:

1. Sınıf belirtmeden nesne oluşturabilmek.
2. Nesne gerçeklemesine diğer nesnelerin bağımlılığını azaltmak.
3. Algoritmik bağımlılığı azaltmak.
4. Gevşek ilişkilendirme.
5. Hizmet kalıpları.
6. Kalıtım yerine parça/bütün ilişkisini öneren kalıplar.
7. Sınıf kodlarını daha rahat değiştirmeye yönelik kalıplar.

Birinci grup ilk bakışta tuhaf gelebilir: bir nesne oluşturmak için kurucusu çağrılır, kurucunun adı da sınıf adıyla aynıdır; sınıf belirtmeden nesne nasıl oluşturulur? Bu sorunun yanıtı ilgili kalıplarda görülecektir. Altıncı grup, kalıtımın ağır bir ilişki olmasıyla ilgilidir: deneyimli tasarımcılar mümkün oldukça kalıtım yerine parça/bütün ilişkileriyle modelleme yapar, ama kalıtımın "cuk oturduğu" yerlerde onu kullanmaktan da kaçınmaz. Yedinci gruptaki kalıplar, örneğin bir metodun imzası değiştiğinde o sınıfa bağımlı diğer sınıflarda değişiklik gerekmeyecek tasarımlara yönlendirir.

Hoca bu sıralamayı bırakmasının nedenini şöyle açıkladı: kalıplar birbiriyle ilişkilidir ve bir kalıbı anlatırken başka bir kalıba başvurmak gerekebilir. O kalıp henüz bilinmiyorsa anlatılan şey havada kalır. Bu tür bağımlılık sorunlarına amaca göre yapılan sıralamada daha sık rastlanmış, kitabın üç kümeli sırasında anlatımın daha rahat olduğu görülmüştür. Kalıpları anlatan diğer kaynaklar da bu sırayı izler.

5. Model-View-Controller (MVC) Kalıbı

5.1 Sorun ve çözüm

Derste işlenen ilk kalıp MVC'dir; Dörtlü Çete de kalıpları anlatmaya bu kalıpla başlar. Sorun ve çözüm şöyledir (ekrandan, 00:36:20):

- Sorun: Aynı veriyi farklı biçimlerde gösterebilmek istiyoruz. Kullanıcıya, verinin nasıl ve ne kadarının gösterileceğini seçebilme gibi olanaklar sağlamak istiyoruz.
- Çözüm: Veriyi, verinin gösterimini ve gösterimin yönetimini ayrı sınıflar şeklinde ele almak.

Bu çözüm, geçen haftaki iyi tasarım tanımının doğrudan uygulamasıdır: problem hangi düzeyde parçalara ayrılıyorsa tasarım da aynı düzeyde parçalara ayrılmalıdır. Tek bir veri vardır ama farklı gösterim biçimleri vardır; gösterim biçimini değiştirebilmek ve seçebilmek de ayrı bir gereksinimdir. Problemde üç ayrı bileşen olduğuna göre tasarımda da bu üç bileşen birebir modellenmelidir.

5.2 Seçim sonuçları örneği

Hoca sorunu bir seçim sonucu örneğiyle somutlaştırdı (ekrandan, 00:40:00). Excel'deki tablo şuydu:

Parti	Oy oranı
A	%35
B	%28
C	%17
D	%10
E	%5
F	%2
G	%2
H	%1

Toplam %100'dür. Bu veri pasta grafik (pie chart), sütun grafik ya da bütün rakamların yazıldığı bir tablo olarak gösterilebilir. Pasta grafikte bütün partiler görünür; ama kullanıcı, uygulamadaki bir düğmeye basarak barajın altında kalan küçük partileri "diğer" adı altında tek dilimde göstermek isteyebilir. Seçim zamanlarında haber kanalları sonuçları il il, ittifak bazında ya da başka biçimlerde göstermek ister; belli bir anda pasta grafikten çubuk grafiğe geçmek gerekebilir. Kullanıcının bütün bunları yapabilmesi istenir.

5.3 Sınıflar ve ilişkiler

Hoca kalıbı Violet UML editöründe çizdi (ekrandan, 00:50:00):

- Model: veriyi simgeler.
- View: verinin çeşitli gösterim biçimleridir.
- Controller: kullanıcı komutlarını alıp işler.

İlişkilerin yönleri önemlidir:

```
Controller ──> View
Controller ──> Model
View       ──> Model
```

View, gösterdiği veriyi bilmek zorundadır, bu yüzden Model'e bağlıdır. Controller View'u bilmek zorundadır: örneğin sandıklar sayıldıkça bir partinin oranı değiştiğinde değişiklik Controller üzerinden yapılır ve View'un haberdar edilip grafiği yeniden çizmesi sağlanır. Bu yüzden Controller'dan hem View'a hem Model'e sahiplik ilişkisi vardır.

Bu ilişkiler, bulunması gereken asgari düzeydir. Model, Controller'dan da View'dan da habersizdir ve olması da gerekmez: hangi açıdan bakılırsa bakılsın model veridir. Modelin kendisini hangi kontrolcülerin yönettiğini bilmesine gerek yoktur, çünkü kontrolcüye "şöyle yap" diye bir istekte bulunması söz konusu değildir.

5.4 Ressam benzetmesi

Ders notlarındaki benzetme bir ressamın bir insanın resmini yapmasıdır. Resmini yaptırmak isteyen, ressama poz veren kişi modeldir (Model). Ressam kontrolcüdür (Controller): model

“şöyle çiz” diyebilir, ama resmi nasıl çizeceğine sonunda ressam karar verir ve fırçayı o vurur. Ressam modeli farklı bakış açılarından ve farklı tarzlarda çizebilir, örneğin yalnız portre ya da tüm vücut; bunlar da görünümlerdir (View). Hocaya göre bu benzetme kalıbın akılda kalmasını epey kolaylaştırır.

5.5 Gereksinimlere göre eklenen ilişkiler

Temel yapıya gereksinimlerin dikte ettiği eklemeler yapılabilir. Diyelim ki veriyi değiştirebilecek tek kaynak Controller değildir; veri sahadaki sensörlerden otomatik olarak toplanmaktadır. Bu durumda verideki değişiklikten Controller ya da View nasıl haberdar olacaktır?

Bir seçenek, Controller’ın modeli belirli aralıklarla (örneğin 5 ya da 10 dakikada bir) yoklaması ve değişiklik algılarsa View’a yeniden çizim komutu göndermesidir. Ancak sınıflar farklı bilgisayarlarda, ağ üzerinden haberleşiyorsa ya da iletişim maliyeti veya gecikmesi yüksekse, gereğinden sık yoklamak iletişim maliyetini artırır ve performansı düşürür. Gereğinden seyrek yoklamak ise değişikliklerden geç haberdar olmak demektir. Değişikliklerin ne sıklıkla olacağı da önceden bilinemeyebilir.

Anlık haberdar olmak önemliyse, Model’den Controller’a bir ilişki çizilir: model “ben değiştim” diye kontrolcüyü haberdar eder. Bu konuda ileride Observer kalıbı görülecektir. Hele ki değişiklikler yalnız bizim kontrolcümüz üzerinden gelmiyorsa bu ok gerekli hale gelir. Bu ok çizildiğinde Controller View’u tazeleyebileceği için Model’den View’a ayrıca ok çizmeye gerek yoktur.

Model’den doğrudan View’a ok çizip Controller’a çizmemenin sakıncası, farklı View türlerinin olmasıdır. BarChart, PieChart ve Table gibi gösterim biçimlerinin ortak özellikleri ve metotları ortak bir View üst sınıfında toplanır, kalıtımla aktarılır, gerekirse ilgili metotlar yeniden yazılır (override). Model yalnız bir BarChart nesnesini haberdar ederse diğer View’lar nasıl haberdar olacaktır? View’ların listesini tutması anlamlı olan sınıf Controller’dır. Controller en az bir ya da daha fazla View ile ilişkidedir; bu, şemada `1..*` nicelik bilgisiyle gösterilir (ekrandan, 01:30:00).

Haberdar edilen View’un Controller’a “ben kendimi tazeledim, sen de diğer View’ların tazelenmesini sağla” demesi de mümkündür. Hocaya göre ise bir işi kaynağından başlatmak en doğrusudur; genellikle tek bir kontrolcü olacağından okun Model’den Controller’a olması daha doğrudur.

Bazı senaryolarda birden fazla kontrolcü gerekebilir. Örneğin bir kelime işlemcide komutlar klavyeyle verilebileceği gibi sesle de verilebilir; bir mimari çizimi döndürmek, yaklaştırmak ve uzaklaştırmak için dokunmatik ekrandaki parmak hareketleri, trackball ya da tablet ve kalem kullanılabilir. Bu durumda View’un alt sınıfları gibi Controller’ın da alt sınıfları olur. Model değişikliklerinin View üzerinden bildirilmesi o zaman daha sakıncalıdır: A kontrolcüsünün View’ları ile B kontrolcüsünün View’ları farklıdır; A’nın bir View’u değişiklik aldığında B’nin View’larının haberdar edilmesi için kardeş sınıfların birbirinden haberdar olması gerekir. Hocanın değerlendirmesi: kalıp basit ve öz görünse de çeşitlemeler devreye girince birçok farklı tartışmaya yol açabilir.

5.6 Yararlar ve çeşitlemeler

Slayttaki yararlar ve çeşitlemeler şunlardır (ekrandan, 01:00:00):

- Yararlar: gösterim, kontrol ve veriyi birbirinden ayırarak
  - bunları çalışma anında değiştirebilmek (örneğin bir View’dan diğerine kolayca geçmek),
  - ilgi alanlarının ayrılmasını sağlamak (her varlık yalnız kendisiyle en ilişkili sorumluluklara sahip olduğu için uyum yükselir),
  - yeniden kullanımı artırmak.
- Çeşitlemeler:
  - Model’in geçirdiği değişiklikleri View’a bildirmesi.
  - Birden fazla View: iç içe (nested), ayrı/tek görevcikte (thread), örtüşen/ayrık.

Yeniden kullanıma hocanın örneği: aynı View’lar farklı problem alanlarında kullanılabilir; örneğin belirli aralıklarla yapılan tahlil sonuçlarının (kolesterol, HDL, LDL, demir gibi) izlenmesi

ya da finans alanındaki değişikliklerin izlenmesi. View, Controller ve hatta Model bu alanlarda doğrudan ya da küçük değişikliklerle yeniden kullanılabilir.

Örtüşen View'lara örnek olarak hoca bir yatırım sepetini verdi: birkaç döviz kuru ve birkaç değerli madenden oluşan bir endeks sepeti izlenmek istenir. Farklı View'lar farklı ama kimi yerlerde kesişen kıymetleri gösterebilir; bir değişiklik olduğunda hepsinin güncellenmesini sağlamanın en kolay yolu yine kontrolcüdür. İç içe View'lara örnek, barajı aşamayanları "diğer" diye gösteren View ile bütün partileri gösteren View'dur: biri ötekinin içinde olabilir. Bu tür durumlar söz konusuysa gerçekleme ayrıntılarında bunlara yönelik eklemeler yapmak gerekir.

5.7 MVC ve Microsoft MVC framework'ü

Hoca, özellikle genç meslektaşların "MVC" deyince Microsoft'un framework'ünü düşündüğünü, oysa ilk düşünülmesi gerekenin Model-View-Controller tasarım kalıbı olduğunu vurguladı. Tasarım kalıpları kitabı 1994'te yayımlanmıştır; Microsoft'un framework'ü çok daha sonra ortaya çıkmıştır. Microsoft bu kalıba dayanan ve onu daha rahat, verimli ve esnek gerçeklemeyi sağlayan bir framework geliştirmiştir. Hocaya göre "MVC'yi biliyorum" deyip yalnız framework'ü bilen kişi, daha önemli bir konu olan tasarım kalıplarını çalışmamış demektir.

5.8 MVC üzerine soru-cevaplar

Soru: Django ile web geliştirirken model ve view sınıfları tanımlıyoruz, ama controller olarak ne kullandığımızı çözemedik. Belki farklı bir kalıp kullanıyoruzdur?

Cevap: Hocanın aklına iki olasılık geldi. Birincisi, kontrolcüye ait işlemler View'a yedirilmiş olabilir. View'da verinin işlenmesine ve düzenlenmesine ilişkin işlevler varsa bunlar, bir refactoring eylemi olarak (Extract Class) yeni oluşturulacak bir kontrol sınıfına taşınabilir; böylece MVC tam anlamıyla kurulur. Kontrol sorumluluklarının View'da olması uyumu düşürdüğü için tercih edilmez. İkincisi, View'u kullanan istemci (client) sınıfı aslında kontrolcü olarak nitelendirilebilir. Kontrolcünün View'dan bağımsız olarak veriyi işlemesi hiç gerekmiyorsa, Controller → View → Model biçiminde dolaylı bir ilişki de olabilir. Ama modeli değiştirebilen işlevler View'dan ayrılacaksa kontrolcünün doğrudan Model ile ilişkili olması yararlı olur. Şu anki gereksinimlerde böyle bir şey yoksa bu ilişki olmayabilir. Hoca ayrıca, ileride doğabilecek bakım masraflarını azaltmak için geriye doğru uyumluluğu koruyarak bir kontrolcü sınıfının ayrılmasının (extract) önerilebileceğini ekledi.

Soru (sohbetten): View kutusunun üstündeki `1..*` işareti, Controller'ın birden fazla View ile ilişkide olduğunu mu gösteriyor?

Cevap: Evet. Bir okun ucunda sayı yazılmamışsa genelde 1 anlamına gelir. `1..*`, bir Controller'ın bir ya da daha fazla View ile (genelde sahiplik ilişkisiyle) ilişkili olduğunu söyler. Bu, Controller'da örneğin `myViews` gibi bir üye alanın bulunduğu, bunun da View türünden bir dizi, jenerik liste, bağlı liste, ağaç ya da map gibi bir veri yapısı olduğu anlamına gelir. Şemada üye alan gösterilmese bile bu çoklu sahiplik ilişkisini modelleyen bir veri yapısının mutlaka var olduğu bilinmelidir.

6. Abstract Factory Kalıbı

6.1 "Factory" adlı kalıplar

Bir kalıbın adında "Factory" geçiyorsa, o kalıpta mutlaka nesne oluşturmaya yarayan bir metot ya da bu sorumluluğun atandığı bir sınıf vardır. Bu türden iki kalıp vardır: Abstract Factory ve ardından görülecek Factory Method.

Abstract Factory'nin amacı, birbiriyle ilişkili ya da aynı aileye ait nesneleri, somut sınıflarını belirtmeden oluşturabilmektir.

6.2 Sorun

Hoca önce Java'dan bir örnek verdi: Swing ile çalışırken GUI bileşenlerinin görünüşü (look and feel) çalışma anında değiştirilebilir; örneğin Metal görünümü. Bu, pencere başlığı, düğme, radyo düğmesi, menü gibi bileşenlerin kozmetik görünüşünü değiştirmeye yöneliktir. İstemci

“Metal görünüme geçmek istiyorum” diyebilmeli, ama bundan sorumlu somut sınıfın hangisi olduğunu bilmemeli, yalnız isteğini iletebilmelidir.

Dörtlü Çete’nin örneği ise pencere yöneticileridir. Bir Linux dağıtımında çalışacak bir uygulama hazırlanıyor ve bir form oluşturulurken pencere, kaydırma çubuğu (scroll bar), radyo düğmesi, onay kutusu (checkbox) çizmek gerekiyor. Uygulamanın farklı pencere yöneticileri altında çalışabilmesi isteniyor: kitaptaki örneklerde Presentation Manager (PM) ve Motif, güncel Linux dağıtımlarında GNOME ve KDE. Bir KDE penceresinde KDE kaydırma çubuğu, KDE radyo düğmesi, KDE onay kutusu olmalı; GNOME penceresinde de GNOME bileşenleri olmalıdır.

İstemci “pencere çiz, kaydırma çubuğu çiz, radyo düğmesi koy” diyebilmeli, o anda hangi pencere yöneticisiyle çalışıldığını ve hangi somut sınıfın (örneğin `KDEWindow`, `KDEScrollBar`) kullanıldığını bilmek zorunda olmamalıdır. Hem bu karmaşıklığı bilmesine gerek yoktur hem de bilirse bu sınıflardaki her değişiklikte istemci kodunu da değiştirmek gerekir.

6.3 Çözüm şeması

Kitaptaki şema şöyledir (ekrandan, 01:26:00):

```
Client ──> WidgetFactory (soyut)
           + CreateScrollBar()  (soyut)
           + CreateWindow()     (soyut)
             △
             ├── MotifWidgetFactory: CreateScrollBar(), CreateWindow()
             └── PMWidgetFactory:    CreateScrollBar(), CreateWindow()

Client ──> Window (soyut)         △── PMWindow, MotifWindow
Client ──> ScrollBar (soyut)      △── PMScrollBar, MotifScrollBar

MotifWidgetFactory - - -> MotifWindow, MotifScrollBar   (oluşturur)
PMWidgetFactory    - - -> PMWindow, PMScrollBar         (oluşturur)
```

İstemci soyut `WidgetFactory` ile ilişkilidir; bu sınıf ve bütün metotları soyuttur. `MotifWidgetFactory` ve `PMWidgetFactory` bunun somut alt sınıflarıdır. Alt sınıftan bir nesne, üst sınıftan nesne bekleyen her bağlamda kullanılabileceği için istemci isteklerini soyut sınıfın soyut metotları üzerinden yollar. İstemci Motif üzerinde çalışacaksa ona bir `MotifWidgetFactory` nesnesi, Presentation Manager üzerinde çalışacaksa bir `PMWidgetFactory` nesnesi verilir. İstemci hangi pencere yöneticisiyle çalıştığını bilmeden “bana pencere ver, düğme ver” der ve ona doğru sınıftan doğru nesne iletilir. Böylece sınıf adı belirtmeden nesne oluşturulmuş olur.

6.4 UML gösterimi üzerine uyarılar

Bu şema Dörtlü Çete’nin kitabından alındığı için UML’in eski bir gösterimiyle çizilmiştir. Kalıtım oku kitapta aşağıya bakan bir üçgendir; bu gösterim varlık-ilişki şemalarındaki “karga ayağı” gösterimiyle karışabilir. Derste ve sınavda güncel standart kullanılmalıdır: kalıtımda ucu açık üçgen üst sınıfı gösterir (MVC örneğinde çizildiği gibi).

Şemada italik yazılmış adlar soyuttur: `WidgetFactory` soyut bir sınıf, içindeki `CreateScrollBar()` soyut bir metottur. Ancak kâğıda elle çizerken italik yazı normal yazıdan ayırt edilemez (hocanın deyişiyle hele yazınız biraz yatıksa ya da çirkinse). Bu yüzden hoca şu gösterimi istedi (ekrandan, 01:30:00):

- Soyut sınıf: sınıf adının üstüne «abstract» stereotipi yazılır. Stereotip, « ve » karakterleri arasına yazılan etikettir ve bir UML bileşeni varsayılan anlamı dışında kullanıldığında ne olduğunu belirtir. Normalde bir sınıf somut kabul edilir; soyut olduğunu belirtmek için «abstract» yazılır.
- Soyut metot: metot imzasının sonuna kıvırcık parantez içinde `{abstract}` yazılır. Bu, metot gövdesi anlamına gelmez; UML standardında bu biçimde etiketlenir.

Örneğin `Insan` soyut bir sınıfsa adının üstüne «abstract» yazılır; `+ görevYap()` soyut bir metotsa `+ görevYap() {abstract}` biçiminde gösterilir. Violet UML’de bu seçenek doğrudan

yoktur; hoca bir arayüz (interface) kutusu koyup «interface» yazısını «abstract» olarak değiştirerek bu etiketi elde etti.

6.5 Neden switch-case değil?

Kalıba başka bir seçenek, örneğin Java look and feel için `setLookAndFeel(...)` gibi bir metodun bir enum parametresi alması ve switch-case ile "case Metal: şunu yap" biçiminde dallanmasıdır. Pencere yöneticileri için de "case GNOME, case Motif" yazılabilir.

> **Hocanın vurgusu:** Kendinizi switch-case kullanırken görürseniz kafanızda alarm zilleri çalsın: mutlaka bir kalıtım ilişkisi kurup çok biçimliliği kullanmanız gerekir.

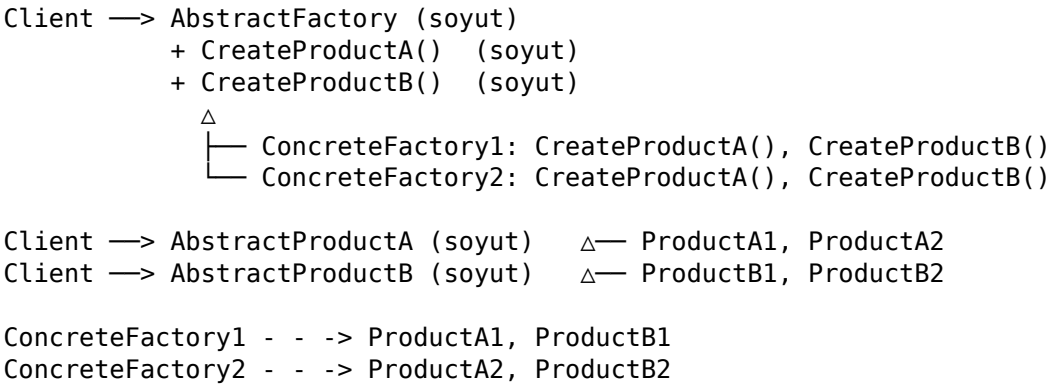
Switch-case'in iki önemli zayıf yönü vardır:

- Çok uzun kod üretir. Her case için benzer uzun kodlar yazılır ve switch'in bulunduğu metot aşırı uzar. Kalıtım hiyerarşisi kurulup her case ayrı bir alt sınıfa denk getirilirse her case'in kodu ilgili sınıfa taşınır; switch'in olduğu kod sadeleşir. Bu taşıma uyumu da yükseltir. Uzun kodu anlamak zordur, bu yüzden bakımı da zorlaşır. Böl ve yönet mantığıyla problemi alt problemlere bölüp her birini kendi metodunda gerçeklemek en iyisidir; bu konu refactoring kısmında tekrarlanacaktır.
- Yeni bir seçenek eklemek zordur. Yeni bir pencere yöneticisi geldiğinde hem enum'a yeni bir değer eklenir hem de switch'in geçtiği her noktaya yeni case ve kodu eklenir. Bu sırada büyük olasılıkla kopyala-yapıştır programlama yapılır; yapıştırılan yerlerle ilgili bir değişiklik gerektiğinde aynı değişiklik her yerde tekrarlanmak zorunda kalınır. Hocanın deyişiyle bunlar bakımı zorlaştıran kâbus gibi olaylardır.

Switch-case tümüyle yasak değildir. İş mantığının (application logic) içinde olmamalıdır, ama örneğin sistem ilklenirken işletim sistemine pencere yöneticisinin ne olduğunu sorup hangi fabrika nesnesinin (Motif, Presentation Manager, KDE) oluşturulacağına karar vermek için bir kez kullanılabilir. Bundan sonra nesneler switch-case'e gerek kalmadan oluşturulur.

6.6 Genel yapı

Kalıbın örnekten bağımsız genel yapısı şöyledir (ekrandan, 01:38:10):



- AbstractFactory: kalıba adını veren soyut sınıf; metotları soyuttur.
- ConcreteFactory: belirli bir ürün ailesinin üyelerini oluşturan somut fabrikalar. Her somut fabrika farklı bir aileyi modeller (örneğin GNOME bileşenleri ailesi, KDE bileşenleri ailesi).
- AbstractProduct: her ürün türü (A ürünü, B ürünü) soyut bir sınıfla modellenir.
- Product: bir aileye ait ürünler, ilgili soyut ürünün alt sınıflarıdır.

İstemci her zaman soyut ürünler üzerinden kodlanır, ama arka planda ona somut bir ürün nesnesi verilir. Hangi aileyle çalıştığını bilmek zorunda kalmaz; bu da ailenin çalışma zamanında bir başkasıyla değiştirilebilmesini sağlar.

6.7 Kalıbın uygulanabileceği anlar

Slayttaki ifadeler Dörtlü Çete'nin kitabından alınmıştır (ekrandan, 01:40:00):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar. Örneğin istemci "pencere ver, kaydırma çubuğu ver" der, bunların nasıl oluşturulduğunu bilmez.
- Bir sistemin farklı ailelerden herhangi birindeki nesneler tarafından yapılandırılması gerektiğinde. Örneğin bir oyun DirectX ya da OpenGL ile çalışacaksa ve OpenGL seçildiyse OpenGL renderer ve shader bileşenleriyle yapılandırılır.
- İlişkili ürün ailelerinden oluşan nesnelerin birlikte kullanılması gerektiğinde.
- Bir ürün ailesindeki nesnelerin bir sınıf kütüphanesi (örneğin public bir API) olarak sunulacağı, ancak gerçeklemeleri yerine yalnızca arayüzlerinin açığa çıkarılacağı durumlarda. Somut sınıflarda `CreateProductA()` çağrıldığında yapılması gereken ek işlemler, hatta çağrılan başka metotlar olabilir; istemci bunların nasıl kodlandığına bağımlı olmaz, yalnız bilmesi gereken public metodu çağırır. İleride görülecek Template Method kalıbı da bu bağlamda kullanılabilir.

Hoca bu cümleleri ezberlemenin verimli olmadığını, yalnız tartışmaya temel olsun diye yazıldığını söyledi.

> **Hocanın vurgusu:** İdeal olan, bir problemi incelerken "bu parçaya şu kalıp yarar" diye hemen ilhamın gelmesidir. Gelmiyorsa kalıp listesine tek tek bakın: bu kalıp problemin bir tarafına yarar mı? Yaramıyorsa geçin. Yarıyorsa seçebilirsiniz, ama devam edin; daha fazla yarayacak başka bir kalıp ya da alt problemlerin farklı kalıplarla çözülmesi daha doğru olabilir. İlk eşleşmede vazgeçmeyin.

7. Abstract Factory Örneği: Üç Boyutlu Oyun Motoru

7.1 Problem

Hocanın kendi hazırladığı örnekte bir üç boyutlu oyun motoru yazılıyor (ekrandan, 01:50:00). Motor hem OpenGL hem DirectX kütüphanelerini desteklemelidir. Kullanılacak işletim sistemine ve ekran kartına göre bunlardan biri otomatik olarak ya da kullanıcı tarafından seçilebilmelidir. Örneğin DirectX tescilli (kapalı kodlu) olduğu için Linux dağıtımlarında desteklenmez; Linux'ta üç boyutlu işlemler için OpenGL kullanılır. Windows'ta da bazı ekran kartları OpenGL'de, bazıları DirectX'te daha iyi başarım verir.

Motorun renderer ve shader gibi bileşenlere ihtiyacı vardır:

- Shader (gölgelendirici): ışık kaynağının yerine göre üç boyutlu varlığın gölgesinin nereye ve nasıl düşeceğini hesaplayan kod kitaplığı.
- Renderer: tel çerçeve olarak tasarlanmış bir varlığın üstüne doku giydirilmesi (texture mapping; örneğin bir insana ten, bir askere zırh ve kask) ve ardından gerçekçi biçimde ekrana çizilmesi (rendering).

Örneklerdeki kodlar iskelet kodlardır; hoca grafik işlemlerinin ayrıntısına girmeden problemin farklı çözümlerine odaklanmak istediğini söyledi. Paket adları her kalıp için aynı biçimdedir, örneğin `dp.abstractFactory.solution2`.

7.2 Çözüm 1: Switch-case kullanımı

İlk akla gelen çözümde kütüphane türü bir enum ile tutulur ve her varyasyon için ayrı sınıflar yazılır (ekrandan, 01:50:00):

```
public enum LibraryType {
   OpenGL, DirectX;
}
public class OpenGLRenderer {
   public void renderOpA() { }
   public void renderOpB() { }
   public void renderOpC() { }
}
public class OpenGLShader {
   public void shadeOpA() { }
   public void shadeOpB() { }
   public void shadeOpC() { }
```

```
}
public class DirectXRenderer {
   public void renderOpA() { }
   public void renderOpB() { }
   public void renderOpC() { }
}
public class DirectXShader {
   public void shadeOpA() { }
   public void shadeOpB() { }
   public void shadeOpC() { }
}
```

Şemada `ApControl` (Application Control, motorun temel sınıfı) dört sınıfın hepsiyle ayrı ayrı ilişkilidir. `ApControl` içinde her olası varyasyon için bir üye alan bulunur: OpenGL renderer nesnesi `o`, OpenGL shader nesnesi `os`, DirectX renderer nesnesi `d`, DirectX shader nesnesi `ds`. Kurucu metotta `library` üye alanı ilklendirilir; bu, işletim sistemine ve grafik sürücüsüne sorarak ya da "işletim sistemi Windows ise DirectX'i seç" gibi kurallarla yapılabilir. Rendering ve shading işlemleri de `library` değerine göre switch-case ile dallanır.

Bu çözümün en önemli zayıf yönü yeni bir kütüphane eklemenin zorluğudur. Windows'un farklı sürümleriyle yeni DirectX standartları gelir ve oyunların sistemin desteklediği en son sürümü desteklemesi istenir. Genel bir DirectX yerine DirectX 10, 11 ve 12 desteklenecekse:

1. Enum'a DirectX 10, 11 ve 12 seçenekleri eklenir.
2. `DirectX10Renderer`, `DirectX10Shader`, `DirectX11Renderer` gibi sınıflar yazılır.
3. `ApControl`'a bunlar için yeni üye alanlar eklenir.
4. Rendering işleminin switch'ine yeni case'ler, shading işleminin switch'ine yeni case'ler eklenir; motorda başka metotlar da varsa onların hepsine de.

Hocanın deyişiyle anlatması bile insanın ağrına gidiyor, yapması nasıl gitmesin. Yeni bir şey eklemek için birçok yerde birçok iş yapmak gerekir; bu bir bakım kâbusudur.

> **Hocanın vurgusu:** Kendinizi switch-case kullanırken görüyorsanız bir yerlerde bir kalıtım ilişkisi kurup çok biçimlilik kullanmanız gerekiyor. Ya da gerekmiyorsa, sorumlulukları yanlış atanmış bir tasarım oluşturmuşsunuz demektir.

7.3 Çözüm 2: Kalıtım kullanımı

İkinci çözüm henüz kalıp kullanmaz, ama bir tasarım ilkesinden yola çıkar: neyin değiştiğini bul ve onu modelle, sarmala. Kütüphanelere bağlı olarak değişen asıl şeyler shader ve renderer bileşenleridir. Bu yüzden soyut `Shader` ve `Renderer` sınıfları oluşturulur ve gereken bütün işlemler soyut metot olarak tanımlanır. Bunlar arayüz (interface) olarak da tanımlanabilirdi: bütün metotları soyut olan ve üye alan gerektirmeyen yapılar arayüz olarak tanımlanabilir. Örnekte soyut sınıf seçilmiştir. Somut `OpenGLShader` ve `DirectXShader` (benzer biçimde `OpenGLRenderer` ve `DirectXRenderer`) sınıfları kalıtımla türetilir ve soyut metotların somut gerçeklemelerini içerir (ekrandan, 02:00:00):

```
package dp.abstractFactory.solution2;
public class ApControl {
   private LibraryType library;
   private Renderer r;
   private Shader s;
   //library nesnesini ilklendirip kütüphane belirle
   public ApControl( ) {
      switch (library){
         case OpenGL:
            r = new OpenGLRenderer();
            s = new OpenGLShader();
            break;
         case DirectX:
            r = new DirectXRenderer();
            s = new DirectXShader();
            break;
```

```
        }
    }
    void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

Şemada `ApControl` yalnız soyut `Shader` ve `Renderer` ile ilişkilidir; soyut metotlar `+ void shadeOpA() {abstract}` biçiminde gösterilmiştir. Enum hâlâ vardır, çünkü sistemin başlangıçta hangi kütüphaneyi kullanacağına karar vermek için gereklidir. `library` üye alanı switch'ten önce bir yerde ayarlanmalıdır: işletim sistemi ve donanımın neyi daha iyi desteklediğini sorgulayan bir kodla ya da kullanıcının oyunun ilk açılışındaki yapılandırma ekranında yaptığı seçimle.

Switch-case artık yalnız kurucudadır ve yalnız hangi türden renderer ve shader nesnesinin oluşturulacağına karar verir. Atama yapıldıktan sonra her işlem soyut tür üzerinden yaptırılır ve arka planda OpenGL mi DirectX mi olduğunu bilmeye gerek kalmaz.

DirectX 10, 11, 12 senaryosu tekrarlandığında: DirectX 10, 11 ve 12 shader ve renderer kardeş sınıfları oluşturulur. Hoca bunun zaten yapılması gereken iş olduğunu vurguladı: yeni bir şey desteklenecekse onun kodu yazılmak zorundadır; önemli olan bunun üzerine yapılması gereken ek işlerin az olmasıdır. Enum'a birer seçenek ve kurucudaki switch'e `r = new DirectX11Renderer(); s = new DirectX11Shader();` gibi birer case eklenir. `doRendering()` ve `doShading()` gibi, oyunun iş mantığını taşıyan metotlarda en ufak bir değişiklik gerekmez. Bu çözüm derli toplu görünür.

7.4 Çözüm 3: Abstract Factory kalıbının kullanımı

Farklı ailelerin (OpenGL, DirectX 10 ve diğerleri) ve her ailede birlikte kullanılan bireylerin (shader ve renderer) olduğu fark edilince Abstract Factory fikri akla gelir. Kalıtım hiyerarşisi bozulmaz: soyut `Shader` ve `Renderer` ile somut alt sınıfları kalır, `ApControl` yine soyut sınıflar üzerinden çalışır. Buna ek olarak:

- Soyut `ComponentFactory` sınıfı, `createRenderer(): Renderer {abstract}` ve `createShader(): Shader {abstract}` metotlarını tanımlar. Bu gösterim, metodun soyut olduğunu ve geriye `Renderer` (ya da `Shader`) türünden nesne döndürdüğünü söyler.
- `OpenGLFactory`, `createRenderer()` çağrıldığında bir `OpenGLRenderer` nesnesi oluşturup döndürür; shader için de `OpenGLShader` döndürür.
- `DirectXFactory`, aynı metotlarda `DirectXRenderer` ve `DirectXShader` nesneleri döndürür.

Aslında `ComponentFactory`'den soyut ürünlere ve somut fabrikalardan kendi somut ürünlerine kesikli çizgili (oluşturma) okları da çizmek gerekir. Hoca bunları Violet UML'de çizmeyi denedi, ama çizgiler birbirini kestiği için şema karmaşıklaştı; bu ilişkilerin sınıf adlarından anlaşılabileceğini söyleyerek çizimleri sildi.

`ApControl`'un yeni kodu şöyledir (ekrandan, 02:08:00):

```
package dp.abstractFactory.solution3;
public class ApControl {
    private ComponentFactory library;
    private Renderer r;
    private Shader s;
    public ApControl( ComponentFactory library ) {
          this.library = library;
          /* Seçenek A: library nesnesini ilklendirerek kütüphane türünü
           * switch-case ile önceki örnekteki gibi belirle
           * Seçenek B: buradaki gibi kurucu metoda parametre olarak ver.
           * Seçenek C: Farklı tür hedef bilgisayarlar için
           * farklı dağıtımlar yap ve alttaki uygun satırın yorumunu kaldır.
          // library = new OpenGLFactory();
```

```
            // library = new DirectXFactory();
            */
            r = this.library.createRenderer();
            s = this.library.createShader();
    }
    void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

Artık kütüphane türlerini belirten enum'a gerek yoktur; `ApControl`'un yalnız ilgili bir fabrikayla ilişkili olması yeter. Fabrika nesnesi kurucuya parametre olarak verilir. Bu nesne bir yapılandırma dosyasından belirlenebilir; kullanıcının seçimiyle yazılan bir yapılandırma dosyası da olabilir. Önceki gibi kurucuda switch-case ile belirlemek de bir seçenektir, ama hocaya göre kalıbın avantajlarını en iyi kullanan seçenek fabrika nesnesinin kurucuya parametre olarak verilmesidir. `doRendering()` ve `doShading()` metotları hiç değişmemiştir.

Slaytta, refactoring konusu görüldükten sonra "Preserve Whole Object" eyleminin yürütülmüş halinin tercih edildiği yazıyordu (ekrandan, 02:10:00). Hoca bunun refactoring'e ait bir konu olduğunu, burada yersiz bir başvuru olduğunu söyleyerek geçti.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın kataloğundaki Preserve Whole Object eylemi, bir nesneden birkaç değer alınıp bunlar ayrı ayrı bir metoda parametre olarak geçirilecekse bunun yerine nesnenin kendisinin geçirilmesini önerir. Bu sayede metot, ileride o nesneden başka bir değere ihtiyaç duyduğunda imzası değişmez.

7.5 Çözüm 2 ile Çözüm 3'ün karşılaştırılması

Hoca, ilk çözümün kötü olduğu açık olduğu için yalnız kalıtımlı çözüm ile kalıplı çözümü karşılaştırdı. Eclipse'te iki `ApControl.java` dosyasını seçip "Compare With > Each Other" ile satır satır karşılaştırdı (ekrandan, 02:11:40):

- Çözüm 2'de `library : LibraryType` üye alanı ve parametresiz `ApControl()` kurucusu vardır; Çözüm 3'te bunların yerini `library : ComponentFactory` alanı ve `ApControl(ComponentFactory)` kurucusu alır.
- Çözüm 2'nin kurucusunda switch-case vardır, Çözüm 3'te yoktur. Switch-case ne kadar azalırsa o kadar iyidir.
- Renderer ve shader alanları ile `doRendering()`, `doShading()` aynıdır.

Kalıtımlı çözümde de switch-case ortadan kaldırılabilir: sistemi tarayıp hangi kütüphanenin kullanılacağını döndüren ayrı bir sınıf yazılabilir ya da bu iş aynı sınıfın başka bir metoduna taşınabilir. Böylece motor sınıfı kendi asıl işine odaklanır. Ama bu, kalıtımlı çözümde ek bir zorlama gerektirir; kalıplı çözümde ise fabrika sınıfı bu sorumluluğun atanacağı çok doğal bir yerdir. Hocaya göre kalıplı çözümün önemli avantajı budur; bunun dışında büyük bir avantajı yoktur. Diğer sınıflar ya yeni eklenmiş ya da kodu değişmemiştir.

Kalıp kullanmak üç yeni sınıf (`ComponentFactory`, `OpenGLFactory`, `DirectXFactory`) getirmiştir; bakım masrafının arttığı düşünülebilir ve kalıtımlı çözüm de tercih edilebilir. Hoca ise nesne türünün belirlenmesi işini ayrı bir sorumluluk olarak ayrı bir sınıfa aktararak uyumu iyileştirdiği için kalıplı çözümü tercih ettiğini söyledi. Bu, geçen haftaki veritabanı örneğine benzer: bağlaşımı biraz artırmak uyumu iyileştirmiştir. Ancak buradaki ek kazanım o örnekteki kadar belirgin değildir, bu yüzden kalıbı seçmek zorunlu değildir.

7.6 Kalıtımın yanlış kullanımı

Kalıtım ters yönde kurulsaydı çok daha kusurlu bir tasarım ortaya çıkardı (ekrandan, 02:20:00). Hoca bunu Violet UML'de karaladı: `ApControl` sanki OpenGL ve DirectX için ayrı sürümler yazılacakmış gibi düşünülüp `OpenGLApControl` ve `DirectXApControl` alt sınıfları oluşturulur. Her biri yine kendi renderer ve shader nesnelerine (`DirectXRenderer`, `DirectXShader`, `OpenGLRenderer`, `OpenGLShader`) birebir sahiplik ilişkisiyle bağlanır.

Bu durumda `doRendering()` ve `doShading()` gibi metotlar iki kardeş alt sınıfta da kopyala-yapıştır kod olarak tekrarlanır. Çok benzer kodun kardeş sınıflarda tekrarlanması hiç doğru değildir: bir değişiklik gerektiğinde aynı değişiklik bütün noktalarda yapılmak zorunda kalınır ve düzgün bir bakım yapılamaz. Ortak kodu üst sınıfa taşımaya kalkmak başka sorunlar çıkarır; belki Template Method gibi bir kalıpla bu da çözülebilir, ama daha düz ayak bir kalıtımlı tasarım ve kalıplı tasarım varken bu yola gitmek işi gereksiz yere karmaşıklaştırır. Hoca, bu kötü tasarımı koda dökmenin küçük bir alıştırma olarak yararlı olacağını, sakıncaların böylece birebir görüleceğini söyledi.

8. Örnekten Çıkarılan Tasarım İlkeleri

Slayttaki "Çıkartılan dersler = tasarıma ait genel kurallar" şöyledir (ekrandan, 02:20:00):

- Neyin değiştiğini bul ve onu sarmala.
  - Böylece bu şeyi soyutlamış olursun.
  - Böylece bu şeyi kara kutu yaklaşımı ile kullanabilirsin.
  - Böylece bu konu ile ilgili değişiklikler sadece bu kısmı etkiler.
  - Örnekte kullanılan yer: hangi grafik kütüphanesinin kullanılacağı bir bilgisayardan başka bilgisayara geçilince değişeceğinden, bu değişim `ComponentFactory` sınıfında soyutlanmıştır.
- Kalıtım ilişkisi yerine parça/bütün ilişkisini tercih et.
  - Kalıtım farklı, sakıncalı bir biçimde kurulabilirdi: `ApControl` üst sınıfından `OpenGLApControl` ve `DirectXApControl` türetilebilirdi. Sonuçta benzer kodun birden fazla yerde tekrarlanması nedeniyle kusurlu bir tasarım ortaya çıkacaktı.
- Gerçeklemelere göre değil, arayüzlere göre tasarlama yap.
  - Böylece kara kutu yaklaşımını kullanabilirsin.
  - Örnekte `ApControl` sınıfı `ComponentFactory` sınıfından bir renderer ve bir shader oluşturmasını istiyor; bunu nasıl oluşturduğu umurunda değil.

8.1 Kara kutu ve beyaz kutu

"Sarmala" derken uygun bir biçimde modellemek kastedilir. Değişen şey sarmalanınca soyutlanmış olur ve kara kutu (black box) olarak kullanılabilir: yalnız sunduğu arayüz, yani public metotları üzerinden, iç ayrıntılarını bilmeden. Tersi beyaz kutudur (white box): kullanmak için iç yapısını bilmek gereken bileşen. Kara kutu erişim kullanıcı ya da programcı için elbette daha kolaydır.

Hocanın örneği televizyondur: televizyon izlemek için tüplü, LCD ya da OLED televizyonların çalışma ilkelerini bilmek gerekmez. Kanal değiştirme ve ses düğmelerini bilmek yeter; bunlar da evrensel sembollerdir. Kara kutu yaklaşımı, bir konudaki değişikliklerin yalnız o kısmı etkilemesini de sağlar. Kötü çözümde grafik kütüphanesi eklemek ya da bilgisayar değiştiğinde kütüphane kararının değişmesi birçok yeri etkiliyordu; kalıplı çözümde bu karar `ComponentFactory`'de soyutlanmıştır ve `ApControl`'a yalnız "bana renderer ver, shader ver" demek kalmıştır.

8.2 "is-a" ilişkisi

Kalıtım bir "aynısıdır" ilişkisidir: insan sınıfından kalıtımla öğrenci sınıfı oluşturulduğunda her öğrenci aynı zamanda bir insandır ("every student is a human"). Bu yüzden kalıtıma "is-a" ilişkisi de denir. Is-a ilişkisinin geçerli olduğu her yerde kalıtım çekinmeden kullanılabilir ve kullanılmalıdır. Bunun dışındaki durumlarda, anlamsız kalmıyorsa parça/bütün ya da sahiplik ilişkileri tercih edilir.

8.3 Arayüze göre tasarım

Burada "arayüz", bir sınıfın ya da varlığın dışarıya sunduğu hizmetler anlamında kullanılır (Java'daki `interface` anahtar sözcüğüyle sınırlı değildir). Doğru tasarımda `ApControl`, `ComponentFactory`'den yalnız "bana renderer ver, bana shader ver" ister. Nesnenin nasıl oluşturulduğunu ya da bunun için fabrikanın hangi alt metotlarının hangi sırayla çalıştırılması gerektiğini bilmesine gerek yoktur; bu tamamen fabrikanın sorumluluğudur.

Hoca, ilk ayrıntılı kalıp olduğu için Abstract Factory'yi üç farklı çözümle işlediğini, diğer kalıpların çoğunda bir kötü çözüm ve bir kalıplı çözüm, bazılarında yalnız kalıbın uygulanışı gösterileceğini belirtti.

Örnek Sorular

1. (derste sözlü, 01:07:51) MVC şemasında View kutusunun yanındaki `1..*` işareti ne anlama gelir?

> **Çözüm (derste verildi):** Controller'ın bir ya da daha fazla View ile (genelde sahiplik) ilişkisinde olduğunu gösterir. Okun bir ucunda nicelik yazılmamışsa genelde 1 anlamına gelir. `1..*` ilişkisi, Controller'da View türünden elemanlar tutan bir veri yapısının (dizi, liste, ağaç, map gibi) bulunduğunu gösterir; şemada bu üye alan gösterilmese bile var olduğu bilinmelidir.

2. (derste sözlü, 01:03:51) Bir web çatısında model ve view sınıfları tanımlıyoruz ama controller'ı göremiyoruz. MVC açısından bu durum nasıl yorumlanır?

> **Çözüm (derste verildi):** İki olasılık vardır. (a) Kontrol işlemleri View'a yedirilmiştir; bu uyumu düşürür. Veriyi işleyen işlevler Extract Class refactoring eylemiyle yeni bir kontrol sınıfına taşınarak MVC tam olarak kurulur. (b) View'u kullanan istemci sınıf kontrolcü rolünü oynuyordur. Kontrolcünün veriyi View'dan bağımsız işlemesi gerekmiyorsa Controller → View → Model dolaylı ilişkisi yeterli olabilir. Gerekiyorsa Controller'dan Model'e doğrudan ilişki kurulmalıdır.

3. (derste sözlü, 02:16:01) Çözüm 3'teki `ApControl` kurucusunda `this.library = library;` ataması yapılmadan doğrudan `library.createRenderer()` yazılabilir mi?

> **Çözüm (derste verildi):** Hocanın yanıtı: atama mutlaka yapılmalıdır. `library` üye alanı ilklendirilmemişse onun bir metodunu çağırmak çalışma zamanında `NullPointerException` aykırı durumuna yol açar. Fabrika nesnesi ya kurucuya parametre olarak verilip atanmalı ya da kurucu içinde (örneğin switch-case ile) belirlenip atanmalıdır.
>
> **Ders sonrası not:** Java'da kurucunun parametresi aynı adlı üye alanı gölgeler. Bu yüzden kurucu içinde `r = library.createRenderer();` yazmak (parametre üzerinden) hata vermez. Ancak `this.library` atanmazsa üye alan `null` kalır ve sınıfın başka bir metodu `this.library` üzerinden fabrikayı kullanmaya kalkarsa `NullPointerException` oluşur. Hocanın uyarısı bu üye alan için geçerlidir.

4. (ekrandan, 02:10:00) "Refactoring konusunu gördükten sonra, Preserve Whole Object eyleminin yürütülmüş hali tercih edilir: Neden? Bize ne kazandırır?"

> **Çözüm (derste verilmedi):** Hoca bu soruyu refactoring konusuna bıraktı. Derste fabrika nesnesinin kurucuya parametre olarak verilmesinin kalıbın avantajlarını en iyi kullanan seçenek olduğu söylendi: `ApControl` kütüphane türünü belirlemekle uğraşmaz, enum ve switch-case ortadan kalkar. Preserve Whole Object ile bağlantısı büyük olasılıkla şudur: kurucuya kütüphane türünü belirten bir değer yerine bu bilgiyi taşıyan bütün nesne (fabrika) verilmekte ve gereken bileşenler o nesneden istenmektedir.

5. (derste sözlü, 02:25:59) Alıştırma: `ApControl`'dan `OpenGLApControl` ve `DirectXApControl` türeten yanlış kalıtımlı tasarımı koda dökün ve sakıncalarını gözleyin.

> **Çözüm (derste verilmedi, hocanın anlatımından):** Her alt sınıf kendi renderer ve shader nesnesine sahip olur ve `doRendering()`, `doShading()` metotları her iki alt sınıfta neredeyse aynı kodla tekrarlanır. Bu metotlarda bir değişiklik gerektiğinde değişiklik her kopyada ayrı ayrı yapılmak zorunda kalınır; yeni bir kütüphane (örneğin DirectX 11) için de yeni bir `ApControl` alt sınıfı ve kodun bir kopyası daha gerekir.

Dersin Toparlaması

Bu derste tasarım kalıbı kavramı tanımlandı: belirli sorunların yalın ve güzel çözümlerinin tarifi. Bir kalıbın isim, sorun, çözüm ve tartışma/sonuçlar bileşenleri, kalıpların yararları (özellikle ekip içinde ortak sözlük oluşturması) ve Dörtlü Çete'nin sınıflamaları (yaratımsal, yapısal, davranışsal; sınıfsal ve nesnesel; yedi gruplu sınıflama) ele alındı. İlk kalıp olarak MVC işlendi. Veri, gösterim ve gösterimin yönetimi ayrılarak gösterimin çalışma anında değiştirilebildiği, ilgi alanlarının ayrıldığı ve yeniden kullanımın arttığı görüldü. Model değişikliklerinin bildirilmesi ve birden çok View ya da Controller gibi çeşitlemelerin tasarımı nasıl etkilediği tartışıldı.

İkinci kalıp, ilk yaratımsal kalıp olan Abstract Factory'dir. Oyun motoru örneğinde switch-case, kalıtım ve kalıp çözümleri karşılaştırıldı. Switch-case'in bakım kâbusuna yol açtığı, kalıtımla değişen bileşenlerin sarmalanmasının işi büyük ölçüde çözdüğü, kalıbın ise nesne türünü belirleme sorumluluğuna doğal bir yer vererek uyumu biraz daha iyileştirdiği görüldü. Örnekten üç genel ilke çıkarıldı: neyin değiştiğini bul ve sarmala, kalıtım yerine parça/bütün ilişkisini tercih et, gerçeklemelere göre değil arayüzlere göre tasarla. Bir sonraki konu Factory Method kalıbıdır. İki kalıbın adında da "Factory" geçtiği için birbirine karıştırılabilirler; hoca aralarındaki farkı Factory Method'dan sonra özellikle vurgulayacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.